

Resumen Teórico: La Clase NP y NP-Compleitud

Computabilidad y Complejidad — UNRC

Basado en el material de Pablo Castro (UNRC)

Índice

1. Introducción	2
2. La clase NP	2
3. Un ejemplo guía: caminos Hamiltonianos	3
3.1. Una solución no determinista para HAMPATH	3
4. Verificadores y certificados: definición alternativa de NP	3
4.1. Equivalencia entre las dos definiciones	4
5. Ejemplos de problemas en NP	4
5.1. HAMPATH	5
5.2. COMP (números compuestos)	5
5.3. CLIQUE	5
5.4. SUBSET-SUM	5
5.5. Problemas que parecen estar afuera de NP	6
6. P vs NP	6
7. Reducibilidad polinomial	6
7.1. Propiedades de la reducibilidad polinomial	7
8. NP-Compleitud	7
9. SAT y el Teorema de Cook–Levin	8
9.1. Fórmulas booleanas y SAT	8
9.2. Formas normales: CNF y 3-CNF	8
9.3. Teorema de Cook–Levin	8
9.4. Idea de la reducción: la tabla de cómputo	8
9.5. Variables proposicionales	9
9.6. Estructura de la fórmula ϕ	9
9.7. Tamaño total de ϕ	10
9.8. Correctitud	10
10.3-SAT es NP-completo	11
11.Reducciones desde 3-SAT	11
11.1. $3\text{-SAT} \leq_P \text{CLIQUE}$	11
11.2. $3\text{-SAT} \leq_P \text{VERTEX-COVER}$	12
11.3. $3\text{-SAT} \leq_P \text{HAMPATH}$	13

12.Otros problemas NP-completos clásicos	13
13.Algunas propiedades útiles de las clases	13
14.Una breve mirada a la complejidad espacial	14
15.Resumen final	14

1. Introducción

En el teórico anterior estudiamos la clase P: los problemas decidibles en tiempo polinomial por una máquina de Turing determinista. La pregunta natural ahora es: *¿hay problemas más difíciles que los de P pero que aún tengan “algo de estructura”?* La respuesta es la clase NP, que reúne a una enorme cantidad de problemas naturales para los cuales no se conoce ningún algoritmo polinomial pero que, sin embargo, exhiben una propiedad común: *una solución, si existiera, puede ser verificada rápidamente.*

Este resumen recorre las herramientas clásicas de esta área:

1. La definición de NP mediante máquinas de Turing no deterministas y mediante verificadores polinomiales.
2. Ejemplos canónicos de problemas en NP: HAMPATH, CLIQUE, SUBSET-SUM, COMP.
3. La relación $P \subseteq NP$ y la conjetura $P \stackrel{?}{=} NP$.
4. Reducibilidad polinomial: la herramienta que permite comparar la dificultad de problemas.
5. NP-completitud: los problemas “más difíciles” dentro de NP.
6. El Teorema de Cook–Levin: SAT es NP-completo.
7. Reducciones clásicas: $SAT \leq_P 3SAT$, $3SAT \leq_P CLIQUE$, $3SAT \leq_P VERTEX-COVER$, $3SAT \leq_P HAMPATH$.
8. Una primera mirada a la complejidad espacial.

Intuición

Hay problemas que parecen “fáciles de chequear pero difíciles de resolver”. Si alguien nos entrega un camino y nos pregunta si es Hamiltoniano, podemos verificarlo en tiempo lineal. Pero *encontrar* ese camino, sin ayuda externa, parece requerir explorar exponencialmente muchas alternativas. La clase NP formaliza esta asimetría entre *decidir* y *verificar*, y la pregunta abierta $P \stackrel{?}{=} NP$ pregunta si esa asimetría es real o ilusoria.

2. La clase NP

Así como definimos P a partir de máquinas de Turing deterministas, definimos NP a partir de máquinas de Turing *no deterministas*.

Definición 2.1 (Tiempo no determinista).

$$NTIME(t(n)) = \{L \mid L \text{ es decidido por una MT no determinista en } O(t(n)) \text{ pasos}\}.$$

Recordemos que el tiempo de una MT no determinista es el largo de la *rama más larga* de su árbol de cómputo.

Definición 2.2 (Clase NP).

$$\text{NP} = \bigcup_{k>0} \text{NTIME}(n^k).$$

Es decir, NP es la clase de los lenguajes decidibles por alguna MT no determinista en tiempo polinomial.

Intuición

Una MT no determinista en tiempo polinomial es “mágica”: en cada paso puede elegir entre varias transiciones, y acepta una entrada si *alguna* secuencia de elecciones lleva a un estado de aceptación. Esto se parece a tener un oráculo que “adivina” la respuesta correcta y luego permite verificarla.

3. Un ejemplo guía: caminos Hamiltonianos

Definición 3.1 (Camino Hamiltoniano). Dado un grafo dirigido $G = (V, E)$, un **camino Hamiltoniano** entre s y t es un camino que parte de s , termina en t y pasa *exactamente una vez* por cada nodo de V .

Definición 3.2 (Lenguaje HAMPATH).

$$\text{HAMPATH} = \{\langle G, s, t \rangle \mid G \text{ tiene un camino Hamiltoniano de } s \text{ a } t\}.$$

3.1. Una solución no determinista para HAMPATH

Describamos cómo una MT no determinista N puede decidir HAMPATH en tiempo polinomial. Dado $\langle G, s, t \rangle$:

1. **Adivinar.** Escribir, de forma *no determinista*, una secuencia de $|V|$ nodos $v_1, v_2, \dots, v_{|V|}$. (Cada elección se realiza “probando todas las alternativas en paralelo”).
2. **Verificar.** Comprobar:
 - Todos los nodos v_i son distintos.
 - $v_1 = s$ y $v_{|V|} = t$.
 - Para cada i , hay un arco $(v_i, v_{i+1}) \in E$.

Si todo se cumple, aceptar; si no, rechazar.

Ambas etapas se hacen en tiempo polinomial, así que $\text{HAMPATH} \in \text{NP}$.

Idea clave

El patrón es general: una MT no determinista para un problema en NP tiene dos fases.

- **Fase 1 (adivinanza).** Genera *no deterministamente* un candidato a solución.
- **Fase 2 (verificación).** Recorre el candidato y revisa, de manera *determinista y polinomial*, si efectivamente resuelve el problema.

4. Verificadores y certificados: definición alternativa de NP

La idea “adivinar y verificar” se puede formalizar sin recurrir a las MTs no deterministas: basta con definir verificadores deterministas que reciben tanto la entrada como un *testigo* extra.

Definición 4.1 (Verificador). Sea A un lenguaje. Una MT determinista V es un **verificador** para A si

$$A = \{w \mid \exists c : V \text{ acepta } (w, c)\}.$$

La cadena c se llama **certificado** (o testigo) de la pertenencia de w a A .

Definición 4.2 (Verificador polinomial). Un verificador V se dice **polinomial** si:

1. V corre en tiempo $O(n^k)$ para algún $k > 0$, donde $n = |w|$.
2. Los certificados son polinomialmente acotados: existe $k' > 0$ tal que

$$A = \{w \mid \exists c : |c| \leq |w|^{k'} \text{ y } V \text{ acepta } (w, c)\}.$$

Intuición

Un certificado es la *evidencia* de que la respuesta es “sí”. Para **HAMPATH**, el certificado natural es el propio camino Hamiltoniano: una vez que tenemos el camino, verificar que lo es se hace recorriendo aristas. Para que esto sea útil, el certificado tiene que ser *corto* (polinomial respecto a la entrada) y *rápido de chequear* (también polinomial).

4.1. Equivalencia entre las dos definiciones

Teorema 4.3 (Caracterización de NP). $L \in \text{NP}$ si y solo si L tiene un verificador polinomial.

Esquema de la demostración. ($\Leftarrow$) **De verificador a mt no determinista.** Supongamos que L tiene un verificador polinomial V con certificados de tamaño $\leq |w|^k$. Construimos una MT no determinista N que sobre la entrada w :

1. Adivina no deterministamente una cadena c con $|c| \leq |w|^k$.
2. Simula V sobre (w, c) y acepta si y solo si V acepta.

Ambas fases son polinomiales, así que N decide L en tiempo polinomial.

($\Rightarrow$) **De mt no determinista a verificador.** Supongamos que L es decidido por una MT no determinista N en tiempo n^k . Construimos un verificador V que recibe (w, c) donde c codifica una secuencia de elecciones no deterministas (una rama del árbol de cómputo). V simula N siguiendo esas elecciones y acepta si la rama termina aceptando. Como N corre en tiempo n^k , una rama se describe en espacio $O(n^k)$ y se simula en tiempo $O(n^k)$. $\square$

Idea clave

Tenemos dos formas equivalentes de pensar a NP:

- **Operacional:** NP son los problemas decidibles en tiempo polinomial por una MT *no determinista*.
- **Declarativa:** NP son los problemas cuyas instancias positivas admiten una *prueba corta* (certificado polinomial) que puede ser verificada en tiempo polinomial.

La segunda visión suele ser la más útil en la práctica: probar que un problema está en NP casi siempre significa exhibir un certificado natural.

5. Ejemplos de problemas en NP

Veamos varios problemas clásicos y argumentemos que están en NP describiendo un verificador polinomial para cada uno.

5.1. HAMPATH

Ya lo vimos: el certificado es el camino Hamiltoniano $p = v_1 v_2 \cdots v_{|V|}$. El verificador chequea:

- Los v_i son nodos distintos de G .
- $v_1 = s, v_{|V|} = t$.
- $(v_i, v_{i+1}) \in E$ para todo i .

Tamaño del certificado: $|V|$ nodos, claramente polinomial. Tiempo del verificador: lineal en $|G|$. Por lo tanto HAMPATH $\in$ NP.

5.2. COMP (números compuestos)

Ejemplo 5.1.

$$\text{COMP} = \{x \mid \exists p, q > 1 : x = p \cdot q\}.$$

Certificado. Un par (p, q) con $p, q > 1$ y $p \cdot q = x$.

Verificador. Chequea $p, q > 1$ y computa $p \cdot q$ (multiplicación entera, polinomial en la cantidad de bits) comparando con x . Como $p, q < x$, tienen menos bits que x , así que el certificado es polinomial. Por lo tanto COMP $\in$ NP.

5.3. CLIQUE

Definición 5.2 (Clique). Dado un grafo no dirigido G , un **clique** es un subgrafo en el cual cada par de nodos está conectado. Un **k -clique** es un clique con exactamente k nodos.

Ejemplo 5.3.

$$\text{CLIQUE} = \{\langle G, k \rangle \mid G \text{ tiene un } k\text{-clique}\}.$$

Certificado. Un subconjunto $c \subseteq V$ con $|c| = k$.

Verificador. Dado $(\langle G, k \rangle, c)$:

- Chequear que los nodos de c pertenecen a V .
- Chequear que entre cualquier par de nodos de c existe un arco.
- Aceptar si ambas condiciones se cumplen, rechazar en caso contrario.

El subconjunto c tiene tamaño polinomial ($\leq |V|$) y la verificación involucra $\binom{k}{2}$ chequeos de aristas, polinomial. Por lo tanto CLIQUE $\in$ NP.

5.4. SUBSET-SUM

Ejemplo 5.4.

$$\text{SUBSET-SUM} = \{\langle S, t \rangle \mid \exists S' \subseteq S : \sum S' = t\}.$$

Por ejemplo, $\langle \{4, 11, 16, 21, 27\}, 25 \rangle \in \text{SUBSET-SUM}$ porque $4 + 21 = 25$.

Certificado. El subconjunto $S' \subseteq S$.

Verificador. Suma los elementos de S' y compara con t ; chequea además que $S' \subseteq S$.

Tamaño del certificado: a lo sumo $|S|$ números, polinomial. Tiempo: lineal. Por lo tanto SUBSET-SUM $\in$ NP.

5.5. Problemas que parecen estar afuera de NP

Observación 5.5. No todo problema tiene un certificado obvio. Considere los complementos:

$$\overline{\text{HAMPATH}} = \{w \mid w \notin \text{HAMPATH}\}, \quad \overline{\text{CLIQUE}} = \{w \mid w \notin \text{CLIQUE}\}.$$

Para mostrar que un grafo *tiene* un camino Hamiltoniano basta un camino; pero para mostrar que *no tiene* ninguno, no se conoce un certificado polinomial natural. Estos problemas pertenecen a la clase coNP, y no se sabe si $\text{NP} = \text{coNP}$.

6. P vs NP

Podemos resumir las dos clases:

- P: lenguajes que se pueden *decidir* rápidamente.
- NP: lenguajes cuya pertenencia se puede *verificar* rápidamente.

Proposición 6.1. $P \subseteq \text{NP}$.

Demostración. Si $L \in P$, hay una MT determinista M que decide L en tiempo polinomial. Definimos un verificador V que ignora el certificado c y simplemente simula $M(w)$. Esto es polinomial y muestra $L \in \text{NP}$. $\square$

Pregunta abierta: $P \stackrel{?}{=} \text{NP}$

¿Es $P = \text{NP}$? Es decir, ¿todo problema cuya solución se puede *verificar* rápidamente se puede también *resolver* rápidamente? Es uno de los grandes problemas no resueltos de la computación; la mayoría de los investigadores conjetura que $P \neq \text{NP}$, pero nadie ha logrado una prueba. *A priori*, la verificabilidad polinomial parece más potente que la decidibilidad polinomial.

7. Reducibilidad polinomial

Para comparar la dificultad relativa de dos problemas se introduce una noción de *reducción* análoga a las que se usan en computabilidad, pero ahora con la restricción extra de que la función reductora debe ser eficiente.

Definición 7.1 (Función polinomialmente computable). Una función $f : \Sigma^* \rightarrow \Sigma^*$ es **polinomialmente computable** si existe una MT que la computa en tiempo polinomial.

Definición 7.2 (Reducibilidad polinomial). Un lenguaje A es **polinomialmente reducible** a un lenguaje B , notado $A \leq_P B$, si existe una función polinomialmente computable $f : \Sigma^* \rightarrow \Sigma^*$ tal que

$$\forall w : w \in A \iff f(w) \in B.$$

Intuición

$A \leq_P B$ significa que “resolver A no puede ser sustancialmente más difícil que resolver B módulo un trabajo polinomial”. Si sabemos resolver B , podemos resolver A traduciendo cada instancia de A a una de B y usando el algoritmo de B .

7.1. Propiedades de la reducibilidad polinomial

Proposición 7.3. Si $A \leq_P B$ y $B \in P$, entonces $A \in P$.

Demostración. Sea M una MT que computa f en tiempo $O(n^k)$ (la reducción) y M_B una MT que decide B en tiempo $O(n^{k'})$. Construyamos una MT N que decide A :

1. Sobre la entrada w , ejecutar $M(w)$ para obtener $f(w)$.
2. Ejecutar $M_B(f(w))$; aceptar si M_B acepta, rechazar si rechaza.

Por correctitud de la reducción, N decide A . Para el tiempo: $|f(w)| \leq |w|^k$ (porque M corre en $O(|w|^k)$ pasos y no puede escribir más), así que el paso 2 tarda $O((|w|^k)^{k'}) = O(|w|^{k \cdot k'})$. Total: $O(|w|^k) + O(|w|^{k \cdot k'})$, polinomial. $\square$

Proposición 7.4 (Transitividad). Si $A \leq_P B$ y $B \leq_P C$, entonces $A \leq_P C$.

Idea. Componer las dos reducciones: la composición de funciones polinomialmente computables es polinomialmente computable (la salida de la primera es polinomial en la entrada, y eso alimenta a la segunda). $\square$

Observación 7.5. También se puede probar (análogamente) que si $A \leq_P B$ y $B \in NP$, entonces $A \in NP$. Las reducciones polinomiales “preservan hacia atrás” la pertenencia a P y a NP .

8. NP-Compleitud

Dentro de NP hay problemas que son “los más difíciles”: si alguien encontrara un algoritmo polinomial para alguno de ellos, automáticamente tendría algoritmos polinomiales para todos los problemas de NP . Esos problemas se llaman *NP-completos*.

Definición 8.1 (NP-compleitud). Un lenguaje L es **NP-completo** si:

1. $L \in NP$.
2. Para todo $A \in NP$, se cumple $A \leq_P L$ (se dice que L es **NP-difícil**).

Proposición 8.2. Si algún problema NP-completo está en P , entonces $P = NP$.

Demostración. Sea L NP-completo con $L \in P$. Para cualquier $A \in NP$, tenemos $A \leq_P L$. Por la Proposición 7.3, como $L \in P$, se sigue $A \in P$. Por lo tanto $NP \subseteq P$, y como ya teníamos $P \subseteq NP$, concluimos $P = NP$. $\square$

Idea clave

Los NP-completos son “el centro nervioso” de NP . Probar $P = NP$ se reduce a encontrar *un* algoritmo polinomial para *cualquiera* de ellos. Recíprocamente, probar $P \neq NP$ se reduce a demostrar que *ninguno* de ellos admite uno.

9. SAT y el Teorema de Cook–Levin

9.1. Fórmulas booleanas y SAT

Definición 9.1 (Fórmulas booleanas). Sea $X = \{x, y, z, \dots\}$ un conjunto numerable de variables. La gramática de las fórmulas booleanas es:

$$\phi, \psi ::= X \mid \neg\phi \mid \phi \wedge \psi \mid \phi \vee \psi.$$

Definición 9.2 (Asignación y satisfacibilidad). Una **asignación** es una función $v : X \rightarrow \{0, 1\}$. Una fórmula ϕ es **satisfacible** si existe una asignación v que la hace verdadera, denotado $v \models \phi$.

Definición 9.3 (Lenguaje SAT).

$$\text{SAT} = \{\langle \phi \rangle \mid \phi \text{ es una fórmula booleana satisfacible}\}.$$

9.2. Formas normales: CNF y 3-CNF

Definición 9.4 (Cláusula, CNF, 3-CNF). Una **cláusula** es una disyunción de literales (variables o sus negaciones):

$$(\ell_1 \vee \ell_2 \vee \dots \vee \ell_n).$$

Una fórmula está en **CNF** (Conjunctive Normal Form) si es una conjunción de cláusulas:

$$c_1 \wedge c_2 \wedge \dots \wedge c_m.$$

Está en **3-CNF** si cada cláusula tiene exactamente 3 literales. El lenguaje **3-SAT** es el problema de satisfacibilidad restringido a fórmulas en 3-CNF.

Ejemplo 9.5. $(x_0 \vee x_1 \vee x_3) \wedge (\neg x_0 \vee \neg x_1 \vee x_2)$ es una fórmula en 3-CNF.

Observación 9.6. Toda fórmula booleana se puede escribir en CNF (aunque, en general, no en 3-CNF sin agregar variables nuevas; lo veremos al reducir SAT a 3SAT).

9.3. Teorema de Cook–Levin

Teorema 9.7 (Cook–Levin). SAT es NP-completo.

La demostración tiene dos partes.

Parte 1: $\text{SAT} \in \text{NP}$. Dada ϕ , el certificado es una asignación v a las variables de ϕ . El verificador evalúa ϕ bajo v en tiempo polinomial (recorre el árbol sintáctico).

Parte 2: **todo** $A \in \text{NP}$ **se reduce a SAT**. Esta es la parte difícil: dado un lenguaje arbitrario $A \in \text{NP}$ decidido por una MT no determinista N en tiempo n^k , hay que construir, en tiempo polinomial, una fórmula ϕ_w tal que

$$w \in A \iff \phi_w \text{ es satisfacible.}$$

9.4. Idea de la reducción: la tabla de cómputo

Si N acepta w , hay una secuencia de configuraciones $C_0 \rightarrow C_1 \rightarrow \dots \rightarrow C_{n^k}$ que empieza en la configuración inicial sobre w y termina en un estado de aceptación. Esta secuencia se puede representar como una **tabla** (o “tablero”) de $n^k \times n^k$ celdas:

C_0 (configuración inicial)				
C_1				
$\vdots$				
C_{n^k} (aceptación)				

- Cada **fila** representa una configuración (cinta + estado + cabezal).
- Hay n^k filas (una por paso de tiempo) y n^k columnas (longitud máxima de cinta usada).
- Cada celda contiene un símbolo de $C = \Gamma \cup \Sigma \cup Q \cup \{\#\}$, donde Γ es el alfabeto de cinta, Σ el alfabeto de entrada, Q los estados, y $\#$ un delimitador.

9.5. Variables proposicionales

Para cada celda (i, j) y cada símbolo $s \in C$, introducimos una variable

$x_{i,j,s}$ que es verdadera $\iff$ la celda (i, j) contiene el símbolo s .

La cantidad de variables es $|C| \cdot n^{2k} = O(n^{2k})$, polinomial.

9.6. Estructura de la fórmula ϕ

$$\phi = \phi_{\text{cell}} \wedge \phi_{\text{start}} \wedge \phi_{\text{move}} \wedge \phi_{\text{accept}}.$$

Cada componente expresa una propiedad de la tabla.

ϕ_{cell} : cada celda contiene exactamente un símbolo

$$\phi_{\text{cell}} = \bigwedge_{1 \leq i, j \leq n^k} \left[\left(\bigvee_{s \in C} x_{i,j,s} \right) \wedge \left(\bigwedge_{\substack{s, t \in C \\ s \neq t}} (\neg x_{i,j,s} \vee \neg x_{i,j,t}) \right) \right].$$

La primera parte dice “hay al menos un símbolo”; la segunda dice “no hay dos símbolos distintos a la vez”. Tamaño: $O(n^{2k})$.

ϕ_{start} : la primera fila es la configuración inicial

$$\phi_{\text{start}} = x_{1,1,\#} \wedge x_{1,2,q_0} \wedge x_{1,3,w_1} \wedge x_{1,4,w_2} \wedge \cdots \wedge x_{1,n+2,w_n} \wedge x_{1,n+3,\sqcup} \wedge \cdots \wedge x_{1,n^k-1,\sqcup} \wedge x_{1,n^k,\#}.$$

Es decir, la fila 1 codifica la cinta inicial con el cabezal en q_0 apuntando al primer símbolo de w , blanqueado a partir de la posición $n+3$, con delimitadores $\#$ a los costados. Tamaño: $O(n^k)$.

ϕ_{accept} : alguna fila contiene el estado de aceptación

$$\phi_{\text{accept}} = \bigvee_{1 \leq i, j \leq n^k} x_{i,j,q_{\text{accept}}}.$$

Tamaño: $O(n^{2k})$.

ϕ_{move} : cada paso es una transición válida

Esta es la parte más delicada. La idea es usar **ventanas de 2×3 celdas**: una ventana de la tabla, centrada en una posición (i, j) , consta de 6 celdas (3 en la fila i y 3 en la fila $i + 1$). Una ventana es *legal* si su contenido es consistente con *alguna* transición posible de N (incluyendo el caso de celdas que no cambian porque están lejos del cabezal).

Teorema 9.8 (Ventanas y ramas de cómputo). Si todas las ventanas 2×3 de la tabla son legales, entonces la tabla describe una rama de cómputo válida de N .

Intuición.

- Si un símbolo está en el centro de una ventana sin ser adyacente a un estado (cabezal), entonces no debe cambiar de una fila a la siguiente: la ventana lo fuerza a permanecer.
- Si una ventana contiene al estado-cabezal en el centro, las posibles “filas inferiores” se restringen a las que corresponden a alguna transición $\delta(q, a)$.

Ejemplo. Supongamos $\delta(q_1, a) = \{(q_1, b, R)\}$ y $\delta(q_1, b) = \{(q_2, c, L), (q_2, a, R)\}$. Entonces las siguientes son ventanas legales:

a	q_1	b
q_2	a	c

a	q_1	b
a	a	q_2

a	a	q_1
a	a	b

Entonces:

$$\phi_{\text{move}} = \bigwedge_{1 \leq i < n^k, 1 \leq j < n^k} (\text{la ventana centrada en } (i, j) \text{ es legal}).$$

Cada ventana se describe como una gran disyunción sobre las configuraciones legales de 6 celdas; como solo se usan 6 variables, cada cláusula tiene tamaño constante. Tamaño total: $O(n^{2k})$.

9.7. Tamaño total de ϕ

$$|\phi_{\text{cell}}| = O(n^{2k}), \quad |\phi_{\text{start}}| = O(n^k), \quad |\phi_{\text{accept}}| = O(n^{2k}), \quad |\phi_{\text{move}}| = O(n^{2k}).$$

Total: $O(n^{2k})$, polinomial. Además la construcción es polinomialmente computable: simplemente recorreremos todas las celdas y emitimos las cláusulas correspondientes.

9.8. Correctitud

- Si N acepta w , existe una rama de cómputo aceptante; codifica una tabla cuya asignación correspondiente satisface ϕ .
- Si ϕ es satisfacible, la asignación describe una tabla en la que: cada celda tiene un único símbolo (ϕ_{cell}), la primera fila es la configuración inicial (ϕ_{start}), las transiciones son legales (ϕ_{move}), y se alcanza un estado de aceptación (ϕ_{accept}). Esa tabla es, por tanto, una rama de cómputo aceptante de N sobre w , así que $w \in A$.

Esto concluye la demostración del Teorema 9.7.

Idea clave

La fuerza del Teorema de Cook–Levin es enorme: una vez probado que SAT es NP-completo, demostrar que un nuevo problema L es NP-completo no requiere repetir todo este argumento. Basta:

1. Mostrar que $L \in \text{NP}$.
2. Reducir polinomialmente algún problema NP-completo conocido (como SAT, 3SAT, etc.) a L .

La transitividad de la reducción polinomial cierra el círculo automáticamente.

10. 3-SAT es NP-completo

Teorema 10.1. 3SAT es NP-completo.

Esquema. (1) $3\text{SAT} \in \text{NP}$. Idéntico al caso de SAT: el certificado es una asignación.

(2) $\text{SAT} \leq_P 3\text{SAT}$. Construiremos, dada una fórmula CNF arbitraria ϕ , una fórmula 3-CNF ϕ' equisatisfacible (es decir, satisfacible si y solo si ϕ lo es). Como la fórmula construida en Cook–Levin se puede escribir fácilmente en CNF, basta con tratar fórmulas en CNF.

Sea una cláusula $C = (x_0 \vee x_1 \vee \dots \vee x_{n-1})$ con n literales. Introducimos $n-3$ variables auxiliares $z_1, z_2, \dots, z_{n-3}$ y la reescribimos como una conjunción de $n-2$ cláusulas de exactamente 3 literales:

$$(x_0 \vee x_1 \vee z_1) \wedge (\neg z_1 \vee x_2 \vee z_2) \wedge (\neg z_2 \vee x_3 \vee z_3) \wedge \dots \wedge (\neg z_{n-3} \vee x_{n-2} \vee x_{n-1}).$$

Equisatisfacibilidad.

- Si la cláusula original C se satisface con algún literal x_i verdadero, podemos asignar $z_1, \dots, z_{i-1}$ a 1 y $z_i, \dots, z_{n-3}$ a 0 para satisfacer las $n-2$ cláusulas nuevas.
- Recíprocamente, si las nuevas cláusulas se satisfacen, no es posible que todos los x_i sean falsos: una inducción simple a lo largo de la cadena de z s muestra que alguna debe ser verdadera.

Tamaño y tiempo. Para una cláusula de tamaño n , generamos $n-2$ cláusulas y $n-3$ variables. Si ϕ tiene tamaño N , la fórmula ϕ' tiene tamaño $O(N)$ y se computa en tiempo polinomial.

Como ya sabemos SAT es NP-completo, por transitividad ($A \leq_P \text{SAT} \leq_P 3\text{SAT}$ para todo $A \in \text{NP}$) concluimos que 3SAT también lo es. $\square$

11. Reducciones desde 3-SAT

A partir de 3SAT podemos probar la NP-completitud de muchos otros problemas. La estrategia es siempre la misma: dada una fórmula en 3-CNF, construir polinomialmente una instancia equivalente del problema objetivo.

11.1. $3\text{SAT} \leq_P \text{CLIQUE}$

Dada $\phi = (a_1 \vee b_1 \vee c_1) \wedge (a_2 \vee b_2 \vee c_2) \wedge \dots \wedge (a_k \vee b_k \vee c_k)$, construimos un grafo G_ϕ y elegimos k (la cantidad de cláusulas):

- Por cada cláusula generamos un *grupo* de 3 nodos, uno por literal.

- Conectamos cada par de nodos con un arco *excepto* si:
 - Pertenecen a la misma cláusula (mismo grupo).
 - Son contradictorios entre sí (uno es x y el otro $\neg x$).

Teorema 11.1. ϕ es satisfacible si y solo si G_ϕ tiene un k -clique.

Idea. ■ Si ϕ es satisfacible, hay una asignación que hace verdadera alguna literal por cláusula. Tomar uno de esos literales por cláusula: forman un k -clique (todos están en grupos distintos y ninguno es contradictorio con otro porque la asignación los hace simultáneamente verdaderos).

- Si G_ϕ tiene un k -clique K , K debe tener *exactamente uno por cláusula* (no puede tener dos del mismo grupo). Como los nodos de K no son contradictorios, asignando 1 a los literales correspondientes obtenemos una asignación que satisface ϕ .

□

La construcción se hace en tiempo polinomial: G_ϕ tiene $3k$ nodos y $O(k^2)$ aristas.

Ejemplo 11.2. Consideremos $\phi = (x_1 \vee x_2 \vee x_3) \wedge (\neg x_1 \vee \neg x_2 \vee \neg x_3) \wedge (\neg x_1 \vee x_2 \vee x_2)$. El grafo tiene 9 nodos, agrupados en 3 grupos de 3. Un 3-clique consiste en elegir un literal verdadero por cláusula, evitando contradicciones; por ejemplo, $\{x_3, \neg x_1, x_2\}$ funciona y corresponde a la asignación $v(x_1) = 0, v(x_2) = 1, v(x_3) = 1$.

Idea clave

Como CLIQUE $\in$ NP (ya lo vimos) y $3SAT \leq_P$ CLIQUE, concluimos que CLIQUE es NP-completo.

11.2. 3-SAT $\leq_P$ VERTEX-COVER

Definición 11.3 (Cubrimiento de vértices). Dado un grafo no dirigido $G = (V, E)$ y un entero $k \leq |V|$, un **cubrimiento de vértices** de tamaño k es un subconjunto $V' \subseteq V$ con $|V'| = k$ tal que cada arco de G tiene al menos un extremo en V' . El lenguaje es

$$\text{VERTEX-COVER} = \{ \langle G, k \rangle \mid G \text{ tiene un cubrimiento de tamaño } k \}.$$

Cubrimiento en NP. El certificado es un subconjunto V' de tamaño k ; el verificador recorre las aristas y comprueba que cada una toca algún vértice de V' . Polinomial.

Reducción $3SAT \leq_P$ VERTEX-COVER. Dada una fórmula ϕ con m variables y ℓ cláusulas (en 3-CNF), construimos un grafo G_ϕ con dos tipos de *gadgets*:

- **Gadget de variable.** Por cada variable x , dos nodos x y $\bar{x}$ conectados por una arista.
- **Gadget de cláusula.** Por cada cláusula, un triángulo con 3 nodos (uno por literal).
- **Conexión.** Cada nodo de un triángulo se conecta con el nodo de variable que tiene el mismo *label* (un literal $\neg x$ del triángulo se conecta con el nodo $\bar{x}$ del gadget de variable).

Buscamos un cubrimiento de tamaño $k = m + 2\ell$.

Teorema 11.4. ϕ es satisfacible si y solo si G_ϕ tiene un cubrimiento de vértices de tamaño $m + 2\ell$.

Idea. (**Total de nodos:** $2m + 3\ell$.) El grafo tiene $2m$ nodos de variable (uno por literal en cada par $x, \neg x$) y 3ℓ nodos en gadgets de cláusula.

($\Rightarrow$) **Satisfacibilidad $\Rightarrow$ cubrimiento.** Dada una asignación que satisface ϕ , para cada par $(x, \neg x)$ tomamos al nodo correspondiente al literal *verdadero*. En cada triángulo elegimos los dos nodos correspondientes a literales *falsos* en la asignación (el restante es uno que la asignación hace verdadero). Esto da $m + 2\ell$ nodos y se chequea que cubren todas las aristas.

($\Leftarrow$) **Cubrimiento $\Rightarrow$ asignación.** Un cubrimiento de tamaño $m + 2\ell$ es *mínimo* para este grafo: cada par de variable requiere al menos 1 nodo, cada triángulo requiere al menos 2. Por lo tanto, debe contener exactamente 1 por par y 2 por triángulo. Asignamos 1 a las variables que aparecen como literal seleccionado en el par. Como los triángulos están cubiertos por 2 de sus 3 nodos, el nodo no seleccionado se conecta con un nodo de variable que también debe estar fuera; eso significa que su literal correspondiente está en el cubrimiento de variable y, por tanto, es verdadero bajo la asignación, satisfaciendo la cláusula. $\square$

11.3. 3-SAT $\leq_P$ HAMPATH

Ya sabemos que HAMPATH $\in$ NP. Para mostrar NP-completitud se reduce polinomialmente 3SAT a HAMPATH: dada una fórmula ϕ en 3-CNF, se construye un grafo G_ϕ que tiene un camino Hamiltoniano de s a t si y solo si ϕ es satisfacible. La construcción usa gadgets de variable (que codifican la asignación) y gadgets de cláusula (que fuerzan al camino a satisfacer cada cláusula); por su complejidad técnica solo describimos la idea, pero el resultado es que **HAMPATH es NP-completo**.

12. Otros problemas NP-completos clásicos

Existe un enorme catálogo de problemas NP-completos. Algunos otros mencionados en los ejercicios son:

- **LPATH:** $\{\langle G, a, b, k \rangle \mid G \text{ contiene un camino simple de longitud } \geq k \text{ de } a \text{ a } b\}$.
- **3-COLOR:** $\{\langle G \rangle \mid G \text{ es coloreable con 3 colores}\}$.
- **SUBSET-SUM:** ya definido; se puede probar NP-completo reduciendo 3SAT.
- **ISO:** ¿son dos grafos isomorfos? Se sabe que está en NP, pero su estatus de NP-completitud es un problema clásico (en años recientes se han presentado algoritmos cuasi-polinomiales, sin que se haya probado NP-completitud ni pertenencia a P).

13. Algunas propiedades útiles de las clases

A modo de cierre del recorrido por P y NP, mencionamos algunas propiedades de clausura que aparecen como ejercicios:

- P es cerrado bajo **unión, intersección, concatenación y complemento**.
- NP es cerrado bajo **unión, intersección y concatenación**. No se sabe si es cerrado bajo complemento (esta pregunta es $NP \stackrel{?}{=} coNP$).

14. Una breve mirada a la complejidad espacial

Cuando analizamos algoritmos, además del tiempo, otra dimensión importante es la **memoria**.

Definición 14.1 (Complejidad espacial). La complejidad espacial de una MT M sobre una entrada de tamaño n es la *cantidad máxima de celdas de cinta* que utiliza durante su ejecución. Se denota $S_M(n)$.

Intuición

Las preguntas que se hacen para el espacio son análogas a las del tiempo: ¿cuánto espacio necesita un algoritmo para resolver un problema dado? ¿Hay clases de complejidad espacial análogas a P y NP? La respuesta es sí: se definen las clases PSPACE, NPSPACE, L, NL y se estudian sus relaciones (por ejemplo, PSPACE = NPSPACE por el Teorema de Savitch). El estudio sistemático de estas clases queda fuera del alcance de este resumen.

15. Resumen final

Las ideas centrales para llevar:

1. **La clase NP.** Definida operacionalmente como $\bigcup_{k>0} \text{NTIME}(n^k)$, o equivalentemente, como la clase de problemas con *verificadores polinomiales*. La caracterización por certificados es la herramienta práctica para probar la pertenencia.
2. **Patrón “adivinar + verificar”.** Para mostrar $L \in \text{NP}$ basta exhibir un certificado polinomial y un algoritmo polinomial de verificación. Ejemplos: HAMPATH (camino), CLIQUE (conjunto de nodos), SUBSET-SUM (subconjunto), COMP (factorización).
3. $P \subseteq \text{NP}$. Todo problema decidable polinomialmente tiene un verificador trivial. La pregunta *abierta* $P \stackrel{?}{=} \text{NP}$ pregunta si la inclusión es estricta.
4. **Reducibilidad polinomial.** $A \leq_P B$ si existe f polinomialmente computable con $w \in A \iff f(w) \in B$. Propiedades clave: *transitividad, preservación de P y NP hacia atrás*.
5. **NP-completitud.** L es NP-completo si $L \in \text{NP}$ y todo $A \in \text{NP}$ cumple $A \leq_P L$. Son “los más difíciles”; resolver *uno* polinomialmente colapsa $P = \text{NP}$.
6. **Cook–Levin.** SAT fue el primer problema NP-completo encontrado. La demostración codifica una rama aceptante de una MT no determinista como una asignación booleana, mediante la conjunción $\phi_{\text{cell}} \wedge \phi_{\text{start}} \wedge \phi_{\text{move}} \wedge \phi_{\text{accept}}$. El tamaño es polinomial ($O(n^{2k})$).
7. **Cadena de reducciones.** $\text{SAT} \leq_P 3\text{SAT} \leq_P \text{CLIQUE}, \text{VERTEX-COVER}, \text{HAMPATH}, \dots$ Cada nueva reducción agrega un NP-completo al catálogo.
8. **El método estándar.** Para probar que un nuevo L es NP-completo:
 - a) Mostrar $L \in \text{NP}$ con un certificado natural.
 - b) Reducir polinomialmente algún NP-completo conocido a L .
9. **Más allá del tiempo: el espacio.** El estudio de la memoria como recurso da lugar a clases análogas (PSPACE, L, NL) que se exploran en los teóricos siguientes.

Intuición

La teoría de NP-completitud le da estructura a un universo aparentemente caótico de problemas difíciles. Miles de problemas naturales — desde planeamiento y scheduling hasta criptografía y verificación de programas — caen dentro de la clase de NP-completos. Saber que un problema es NP-completo es a la vez una *mala noticia* (no se conoce algoritmo polinomial) y un *conocimiento útil* (no perder tiempo buscando uno, recurrir a heurísticas, aproximaciones o restricciones del problema). Por eso, NP-completitud no es un final triste sino una herramienta de diagnóstico fundamental.